

Static Attributes and Methods and Object Oriented Design

CSCI 1101

2024-04-25

The Object Oriented Programming Paradigm

Recall that in object oriented programming the main tools that we have for making programs are:

classes: mutable types consisting of attributes and methods,

objects: instances of classes, each having its own state but sharing a common interface,

attributes: the private variables of an object where it stores its state,

methods: a collection of functions constituting an interface to an object, allowing us to “send it messages” to request that it perform some actions,

inheritance: a relationship between classes allowing us to share and extend functionality in a controlled way.

Object-Orient All the Things!

In languages that force you to program in an object-oriented way, everything must fit into this paradigm somehow.

In particular:

- *Variables are attributes.*
- *Functions are methods.*

Variables are Attributes

Consider the following class for squares:

```
class Square :  
    def __init__(self , size) :  
        self.size = size  
        self.sides = 4          # 0-0 all the things!  
  
    def get_sides(self) :      # 0-0 all the things!  
        return self.sides  
  
    def perimeter(self) :  
        return self.sides * self.size
```

All Variables are Attributes?

Consider an instance:

```
> s = Square(10)
> s.sides
4
> s.sides = 5
> s.perimeter()
50
```

This is weird because:

- All squares should have the same number of sides.
- A square should not be free to change its number of sides.
- Squares have four sides whether or not we have created any squares.

Some Variables are Static Attributes

In the jargon of object oriented programming, a **static attribute** is just a variable in the namespace of a class:

```
class Square :  
    sides = 4                # a static attribute  
  
    def __init__(self , size) :  
        self.size = size  
  
    def get_sides(self) :  
        return Square.sides    # refers to a static attribute  
  
    def perimeter(self) :  
        return Square.sides * self.size
```

This way it is an “attribute” not of any particular *instance*, but of the *class* in general.

Functions are Methods

So we've solved the problem for the `sides` attribute:

```
> s = Square(10)
> s.sides = 5
> s.perimeter()
40
> s.get_sides()
4
```

But what about for the `get_sides` method?

All Functions are Methods?

The result returned by the `get_sides` method doesn't depend on *which* square we call it on.

Its behavior depends on information stored in the static attribute `Square.sides`, not information stored in an attribute of any particular square.

So we shouldn't need to create any squares to learn how many sides a square has.

Some Functions are Static Methods

In the jargon of object oriented programming, a **static method** is just a function in the namespace of a class:

```
class Square :  
    sides = 4                # a static attribute  
  
    def get_sides() :        # a static method: no self parameter!  
        return Square.sides  
  
    def __init__(self , size) :  
        self.size = size  
  
    def perimeter(self) :  
        return Square.sides * self.size
```

This way it is a “method” not of any particular *instance*, but of the *class* in general.

Calling Static Methods on Instance Objects

We can call a static method the same way we call any function:

```
> Square.get_sides()
```

```
4
```

If we want to also be able to call a static method on class instance objects then we annotate it with a **decorator** to tell python *not* to pass it a `self` argument:

```
@staticmethod          # static method decorator
def get_sides() :
    return Square.sides
```

This lets us write:

```
> s = Square(10)
```

```
> s.get_sides()
```

```
4
```

TL;DR

Static attributes and methods
are just ordinary variables and functions
in the namespace of a class.

The Right Tool for the Job

In languages that force you to program in an object-oriented way, every problem solving strategy must be built from the OOP canon.

For some problem domains this is a good fit, for others, not so much.

There is a risk of starting to believe that all tools are hammers.

You should choose the right tool for the job.

Where OOP Shines

Generally, OOP is a good fit when we have a collection of entities that we want to “choreograph” to carry out a task by manipulating and reporting their internal states.

Examples include:

- mutable data structures (lists, sets, dictionaries)
- drawing libraries
- simulation programs

Issue: Bidirectional Relations

For some relationships between entities, the *attribute* relation isn't the best fit.

Suppose we have a class to represent a person, with attributes for familial relationships:

```
class Person :  
    def __init__(self , name , parents) :  
        self.name = name  
        self.parents = parents  
        self.children = set()  
    def __repr__(self) :  
        return self.name  
    def add_child(self , child) :  
        self.children.add(child)
```

Issue: Bidirectional Relations

If we want to a `Person` instance to be aware of both their parents *and* their children then we need to duplicate the same information in two places:

- in the `children` attribute of the parents,
- in the `parents` attribute of the child.

This risks the information becoming inconsistent if we don't make the same change in both places.

```
> parent_1 = Person('parent_1' , set())  
> parent_2 = Person('parent_2' , set())  
> child_1 = Person('child_1' , {parent_1 , parent_2})  
> parent_1.add_child(child_1)
```

This problem is avoided in the *relational* style of programming.

Issue: Class-Independent Interfaces

Sometimes we have different classes that each implement a method independently rather than by inheritance:

```
class Bat(Animal) :  
    def fly(self) :  
        pass
```

```
class Bird(Animal) :  
    def fly(self) :  
        pass
```

Inheritance doesn't let us specify “an animal that can fly”.

This problem is avoided in languages that allow us to specify an *interface* independent of its *implementation*.

Issue: Inheriting Lost Functionality

Sometimes one class is derived from another but instead of (or in addition to) *adding* functionality, some functionality is removed.

```
class Penguin(Bird) :  
    def fly(self) :  
        return "I can only fly underwater"
```

OOP does not generally specify a way to deal with this scenario.

TL;DR

OOP is a useful tool.

But you should use the right tool for the job.

To Do

Finish *Homework 12: More Object-Oriented Programming* on CodeRunner in by next Thursday.

Finish *Project 4: Bioinformatics* on CodeRunner in by the end of the semester (May 8).